

AKSHAT TALWAR

talwara1@uci.edu | linkedin.com/in/akshat-talwar14 | github.com/AkshatTalwar | 949-992-5845

EDUCATION

University of California, Irvine | B.S. Computer Science

Sept 2022 - June 2026 | GPA: 3.87

Relevant Coursework: DSA (B-trees, hash maps, graph traversal, dynamic programming), Advanced Python Development, Data Mining, Discrete Mathematics

Extracurriculars: Undergrad Teaching Assistant (Python), UI/UX and Web Design Lead - Student Government UCI

PROFESSIONAL EXPERIENCE

Amazon Web Services (AWS)

East Palo Alto, CA

Software Development Engineering Intern

June 2025 – Sept 2025

- Built an ETL pipeline in Java to pull SWF ActivityTask events from the RDS control-plane service and load them into a structured timing and retry schema, processing 100K+ workflow records daily for latency analysis.
- Added 8 new context fields to the workflow analytics schema, which let the team compare p50/p90/p99 latency across workflow steps, regions, and database engine versions, something the old orchestration-only data couldn't show.
- Cut repeated Redshift query load by over 90% with a scheduled EventBridge and Lambda pipeline that pre-aggregates TB-scale latency and retry data into JSON files stored in S3.
- Built a React dashboard that surfaced precomputed metrics through an existing REST API, cutting RDS workflow debugging time from about 45 minutes to about 3 minutes and removing the need for manual SWF and CloudWatch log correlation.

Tech Mahindra

Bedminster, NJ

Software Development Engineering Intern

June 2024 – Sept 2024

- Enabled end-to-end testing of a fraud-scoring prototype with no available real transaction data, by generating a synthetic dataset of 500K+ transactions and building a FastAPI backend that integrated an existing TensorFlow machine learning model behind validated REST endpoints.
- Built a data validation layer (Pydantic schema enforcement) as a feature-integrity gate ahead of the fraud-scoring model, rejecting malformed transactions before they reached model inference.
- Profiled customer-history query performance using EXPLAIN ANALYZE, identified a full-table scan as the bottleneck, and cut p99 latency by 77% (from 350 ms – 80 ms) by adding a composite index on (customer_id, transaction_time).
- Ensured the backend, validation, and query optimizations were verifiable and repeatable, by writing unit and integration tests run against an isolated Dockerized PostgreSQL instance.

Grant Thornton

Delhi, India

Software Development Engineering Intern

June 2023 - Sept 2023

- Built a blob-triggered Azure Function in C#.NET 8 to automate manual review of 100-300 ledger files per day for the audit team, routing approved/rejected files to separate Blob Storage containers, with failure events published to Service Bus using correlation IDs for traceability.
- Diagnosed latency spikes on large files (15-20MB+) using existing Application Insights telemetry, tracing the root cause to full in-memory file loading; refactored the function to stream rows instead, reducing processing time from approximately 30 seconds to 12 seconds.
- Wrote unit tests using xUnit and Moq for validation and routing logic, mocking Blob Storage and Service Bus dependencies, and added them to the team's existing Azure DevOps CI/CD pipeline.

PROJECTS

ZotPool – Real-Time Ridesharing Platform (UCI Hackathon Runner-up 2024)

Next.js, TypeScript, Node.js, PostgreSQL, Redis, Google Maps API, OpenAI API, Docker, AWS Lambda

- Designed a geospatial matching engine with proximity-based scoring and preference weighting to match riders in real time — integrating GMaps API and OpenAI's LLM API, built end-to-end solo.
- Designed the Node.js and PostgreSQL backend with B-tree indexing and Redis caching to handle concurrent ride requests, optimizing query performance under real-time load.
- Implemented WebSockets for live ride status updates and trip coordination, deploying the backend on AWS Lambda with Docker and automating deployments via GitHub Actions CI/CD.

Inelastic Search – Search Engine & Web Crawler

(Python, TF-IDF, PageRank, NLTK, Transformers, Docker)

- Built a web crawler and implemented a map-reduce-style K-way merge strategy for partial inverted index construction, enabling 50K+ page indexing on memory-constrained hardware by merging sorted partial indices without loading full datasets into memory.
- Applied data science techniques (TF-IDF scoring, PageRank-weighted ranking) to build an inverted index, achieving sub-300ms query latency across a 50K+ page corpus and supporting Boolean query operators.
- Designed modular, independently scalable components for crawling, indexing, and query processing, enabling isolated testing and extensible ranking algorithm.

TECHNICAL SKILLS

Languages: Python, Java, C++, C#, TypeScript, SQL

Frameworks: FastAPI, TensorFlow, Pydantic, .NET 8, Node.js, React, Next.js

Cloud & Infrastructure: Azure (Functions, Service Bus, Blob Storage, Application Insights, DevOps), AWS (Lambda, S3, EventBridge, Redshift), Docker

Databases & Analytics: PostgreSQL, MySQL

Testing & CI/CD: xUnit, Moq, GitHub Actions, Azure DevOps CI/CD, REST APIs, WebSockets

AI/ML & NLP: OpenAI API, PyTorch, TF-IDF, PageRank, NLTK, Transformers